

CSCE 763: Digital Image Processing

Homework #3

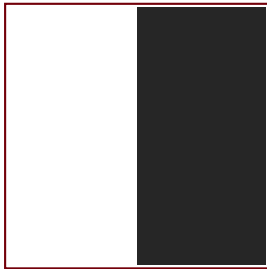
Instructor: Spring 2026
University of South Carolina
Solutions by: J.C. Vaught

Due: 1:15 pm EST, Tuesday, Feb 17

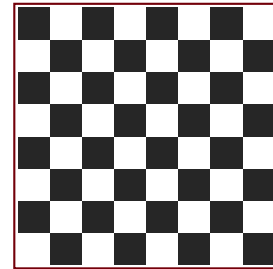
Table of Contents	
Problem 1: Histogram and Blurring (20 pts)	2
Problem 2: Spatial Filtering (20 pts)	5
Problem 3: Averaging Mask and Bar Merging (20 pts)	8
Problem 4: Averaging and Laplacian Order (20 pts)	11
Problem 5: Convolution Filtering (20 pts)	12

Problem 1: Histogram and Blurring (20 pts)

Two $N \times N$ images shown below are quite different, but their histograms are the same. Suppose that each image is blurred with a 3×3 averaging mask. Please ignore the low quality of the images. You can assume all the black regions have a uniform intensity of 0, and the white regions have a uniform intensity of 255.



(a) Half-and-half



(b) Chessboard

- Would the histogram of the blurred images still be equal? Explain.
- If your answer is no, sketch the two histograms.
- Bonus question:** Discuss how the histogram of the blurred image (b) is affected by different image size N . (5 pts)

Step 1: Analyze the Original Histograms

Both images have $N^2/2$ pixels at 0 and $N^2/2$ at 255, so their histograms are identical two-impulse distributions.

Step 2: Analyze Blurring of Image (a) — Half-and-Half

Interior pixels on either side have all 9 neighbors the same color, so they remain at 0 or 255. Only pixels near the vertical split are mixed. For the dominant mixed cases, the 3×3 window has either 3 white + 6 black or 6 white + 3 black pixels, so the blurred values are

$$\frac{3}{9} \cdot 255 = 85, \quad \frac{6}{9} \cdot 255 = 170.$$

So image (a) keeps two strong spikes at 0 and 255, plus much smaller intermediate mass concentrated near 85 and 170 (not centered at 128).

Step 3: Analyze Blurring of Image (b) — Chessboard

For a checkerboard with finite square size, a 3×3 window can contain different counts of white pixels depending on its location relative to square boundaries/corners. If the window contains k white pixels, the output is

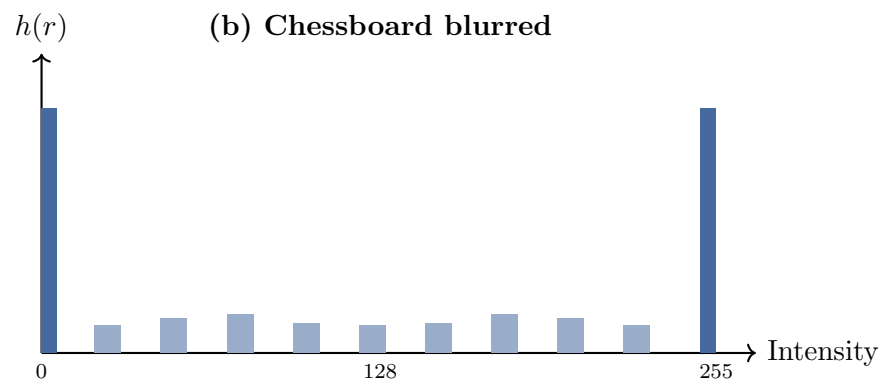
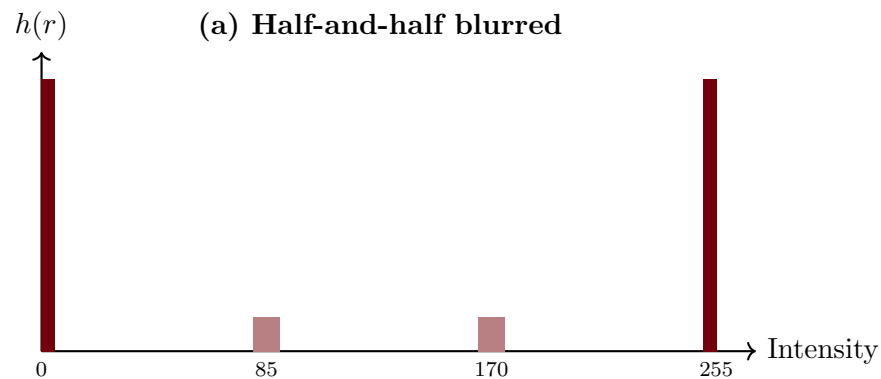
$$r_k = \frac{k}{9} \cdot 255, \quad k = 0, 1, \dots, 9.$$

Therefore image (b) has more gray-level variation than image (a), because many different local white/black combinations appear in 3×3 neighborhoods.

Part (a): Are the Histograms Still Equal?

No. Image (a) retains most pixels at 0 and 255 with a thin strip of mixed values, mainly near 85 and 170. Image (b) exhibits multiple intermediate levels from different 3×3 white/black counts. The histograms differ because the 3×3 averaging mask depends on spatial arrangement, not only on the global counts of 0 and 255 pixels.

Part (b): Sketch of the Two Blurred Histograms



Part (c) Bonus: Effect of Image Size N on Chessboard Histogram

As N increases, the fraction of pixels near square boundaries becomes small compared to the total number of interior pixels. Interior pixels remain near pure black or pure white after averaging, while only boundary pixels produce intermediate values. Since the boundary-pixel fraction tends to zero as $N \rightarrow \infty$, the histogram approaches two dominant peaks at 0 and 255. For smaller N , boundary effects are proportionally larger, so intermediate levels are more visible.

Results

(a) No, the histograms of the blurred images are not equal. Image (a) keeps large spikes at 0 and 255, with mixed values mainly near 85 and 170. Image (b) has broader intermediate-level variation because different 3×3 neighborhoods contain different white/black counts.

(b) See histogram sketches above. Image (a) is strongly concentrated at the extremes with two minor intermediate components. Image (b) has strong extremes plus a richer spread of intermediate values.

(c) Bonus: As $N \rightarrow \infty$, the boundary-affected fraction $\rightarrow 0$, so the blurred chessboard histogram is dominated by peaks at 0 and 255. For smaller N , boundary mixing contributes more intermediate gray levels.

Problem 2: Spatial Filtering (20 pts)

Consider spatial filtering.

- (a) Suppose that you filter an image $f(x, y)$, with a spatial filter mask $w(x, y)$, using convolution, as defined in Eq. (3.4-2), where the mask is smaller than the image in both spatial directions. Show the important property that, if the coefficients of the mask sum to zero, then the sum of all the elements in the resulting convolution array (filtered image) will be zero also (you may ignore computational inaccuracies). Also, you may assume that the border of the image has been padded with the appropriate number of zeros. (10 pts)

$$w(x, y) \otimes f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t) \quad (3.4-2)$$

- (b) Would the result to (a) be the same if the filtering is implemented using correlation, as defined in Eq. (3.4-1)? (10 pts)

$$w(x, y) \odot f(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t) \quad (3.4-1)$$

Part (a): Convolution with Zero-Sum Mask

Let $g(x, y) = w(x, y) \otimes f(x, y)$ denote the filtered image. We want to show that if $\sum_{s=-a}^a \sum_{t=-b}^b w(s, t) = 0$, then $\sum_x \sum_y g(x, y) = 0$.

Step 1: Write the Sum of All Output Pixels

$$\sum_x \sum_y g(x, y) = \sum_x \sum_y \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t)$$

Step 2: Exchange the Order of Summation

Since all sums are finite, we can exchange the order:

$$\sum_x \sum_y g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) \left[\sum_x \sum_y f(x - s, y - t) \right]$$

Step 3: Evaluate the Inner Sum

Consider the inner sum $\sum_x \sum_y f(x - s, y - t)$. Substituting $u = x - s$, $v = y - t$ (a change of summation index), and noting that f is zero-padded (i.e., $f(u, v) = 0$ outside the image), the summation range covers all positions where f is nonzero regardless of the shift (s, t) . Therefore:

$$\sum_x \sum_y f(x - s, y - t) = \sum_u \sum_v f(u, v) = S_f \quad (\text{constant, independent of } s, t)$$

where S_f is the sum of all pixel values in the original image.

Step 4: Factor and Conclude

Substituting back:

$$\sum_x \sum_y g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) \cdot S_f = S_f \cdot \underbrace{\sum_{s=-a}^a \sum_{t=-b}^b w(s, t)}_{= 0 \text{ (given)}} = 0 \quad \blacksquare$$

Part (b): Correlation with Zero-Sum Mask

For correlation, $g'(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$. The same argument applies:

$$\sum_x \sum_y g'(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) \left[\sum_x \sum_y f(x + s, y + t) \right]$$

With the substitution $u = x + s$, $v = y + t$, the inner sum is again S_f (independent of s, t):

$$\sum_x \sum_y g'(x, y) = S_f \cdot \sum_{s, t} w(s, t) = S_f \cdot 0 = 0 \quad \blacksquare$$

Yes, the result is the same for correlation. The sign change in the index ($x - s$ vs. $x + s$) does not affect the argument because the inner sum over a shifted version of f always equals S_f under zero-padding.

Results

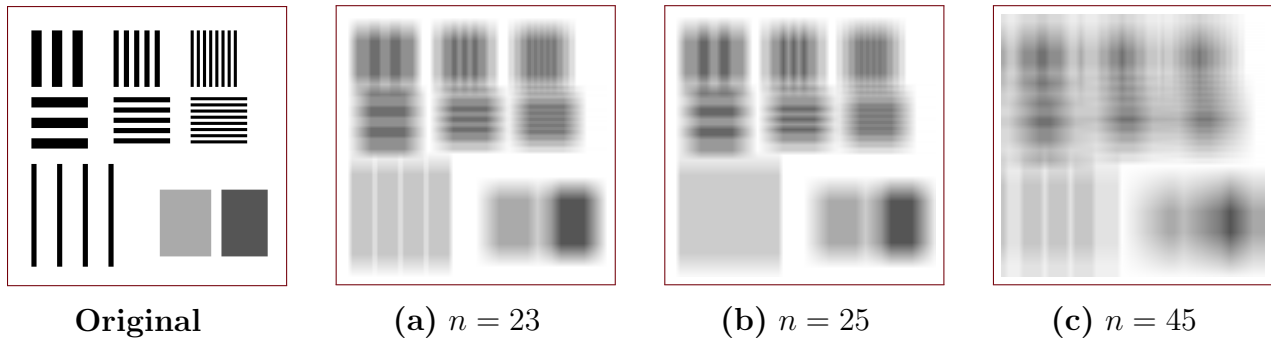
(a) If the mask coefficients sum to zero, i.e., $\sum_{s,t} w(s,t) = 0$, then the sum of all elements in the convolution output is:

$$\sum_{x,y} g(x,y) = S_f \cdot \sum_{s,t} w(s,t) = 0 \quad \blacksquare$$

(b) Yes, the result is identical for correlation. The proof is the same because the inner summation $\sum_{x,y} f(x+s, y+t) = S_f$ is independent of the shift direction. $\blacksquare$

Problem 3: Averaging Mask and Bar Merging (20 pts)

The three images shown were blurred using square averaging masks of sizes $n = 23$, 25 , and 45 , respectively. The vertical bars on the left lower part of (a) and (c) are blurred, but a clear separation exists between them. However, the bars have merged in image (b), in spite of the fact that the mask that produced this image is significantly smaller than the mask that produced image (c).



Explain the reason for this. The vertical bars are 5 pixels wide, 100 pixels high, and their separation is 20 pixels. (Hints: you can simplify the problem by considering a single scan line through the bars in the image.)

Step 1: Simplify to a 1D Problem

Following the hint, consider a single horizontal scan line through the bars. The 1D intensity profile is periodic with bar width 5 pixels (intensity 255), gap width 20 pixels (intensity 0), and **period** $T = 5 + 20 = 25$ **pixels**. The $n \times n$ averaging mask reduces to a 1D moving average of width n .

Step 2: Effect of a Moving Average on a Periodic Signal

A 1D moving average of width n computes:

$$g(x) = \frac{1}{n} \sum_{k=0}^{n-1} f(x - k)$$

This sums n consecutive pixels. For a periodic signal with period T , the key observation is: if n is an exact multiple of T , then the window always contains the same number of “on” (bar) and “off” (gap) pixels regardless of position, making the output **constant**.

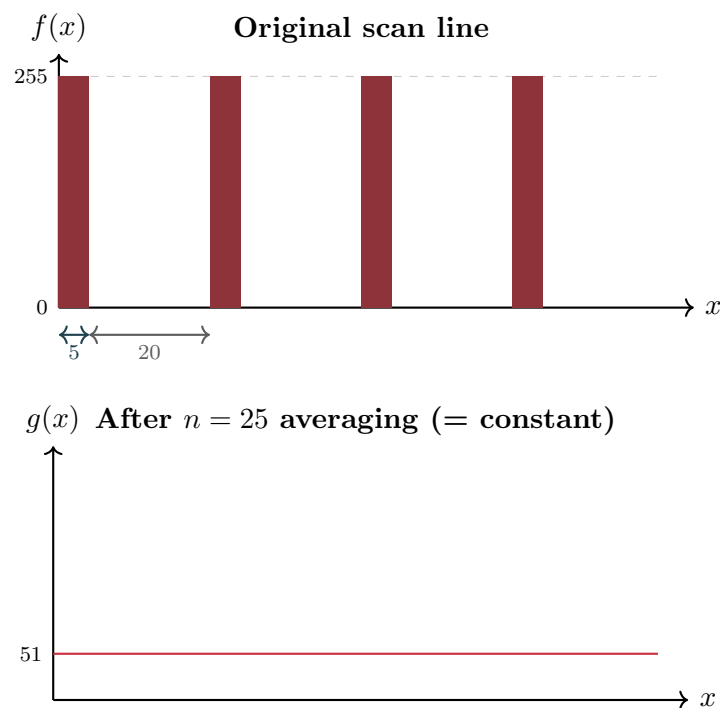
Step 3: Why $n = 25$ Causes Complete Merging

Since $T = 25$ and $n = 25 = 1 \times T$, the averaging window spans **exactly one full period**. At every position, the window contains exactly 5 bar pixels (at 255) and 20 gap pixels (at 0):

$$g(x) = \frac{5 \times 255 + 20 \times 0}{25} = \frac{1275}{25} = 51 \quad (\text{constant})$$

The output is a uniform gray value of 51 along the entire scan line. All spatial variation is destroyed, and the bars **merge completely**.

Visualization: 1D Scan Line Analysis



Step 4: Why $n = 23$ and $n = 45$ Preserve Separation

$n = 23 < T$: The window is smaller than one period, so the number of bar pixels it covers varies with position, preserving the intensity modulation.

$n = 45 \neq kT$: Since $45 = 25 + 20$ is not a multiple of T , the window alternates between covering 1 and 2 bars. The output is more blurred but spatial variation persists.

Results

The bars merge completely when $n = 25$ because the averaging mask width **exactly matches the period** of the bar pattern ($T = 5 + 20 = 25$ pixels). When $n = T$, the moving average window always contains the same number of bar and gap pixels at every position, producing a constant output and destroying all spatial variation. For $n = 23 < T$ and $n = 45 \neq kT$, the window size does not match the period, so the number of bar pixels in the window varies with position, preserving the intensity modulation and keeping the bars distinguishable. ■

Problem 4: Averaging and Laplacian Order (20 pts)

In a given application an averaging mask is applied to input images to reduce noise, and then a Laplacian mask is applied to enhance small details. Would the result be the same if the order of these operations were reversed?

Step 1: Express Both Operations as Convolutions

Let w_{avg} denote the averaging mask and w_{lap} denote the Laplacian mask. Both are linear spatial filters, so both operations are convolutions:

$$g_1 = w_{\text{lap}} \otimes (w_{\text{avg}} \otimes f) \quad (\text{average first, then Laplacian})$$

$$g_2 = w_{\text{avg}} \otimes (w_{\text{lap}} \otimes f) \quad (\text{Laplacian first, then average})$$

Step 2: Apply Associativity and Commutativity of Convolution

Convolution is both **associative** and **commutative**:

$$w_1 \otimes (w_2 \otimes f) = (w_1 \otimes w_2) \otimes f = (w_2 \otimes w_1) \otimes f = w_2 \otimes (w_1 \otimes f)$$

Applying this to our operations:

$$g_1 = w_{\text{lap}} \otimes (w_{\text{avg}} \otimes f) = (w_{\text{lap}} \otimes w_{\text{avg}}) \otimes f$$

$$g_2 = w_{\text{avg}} \otimes (w_{\text{lap}} \otimes f) = (w_{\text{avg}} \otimes w_{\text{lap}}) \otimes f$$

Since $w_{\text{lap}} \otimes w_{\text{avg}} = w_{\text{avg}} \otimes w_{\text{lap}}$ (commutativity), we have:

$$g_1 = g_2$$

Results

Yes, the result is the same regardless of the order. Both the averaging mask and the Laplacian mask are linear, shift-invariant operators (i.e., convolutions). Since convolution is associative and commutative:

$$w_{\text{lap}} \otimes (w_{\text{avg}} \otimes f) = w_{\text{avg}} \otimes (w_{\text{lap}} \otimes f)$$

The mathematical output is identical. ■

Problem 5: Convolution Filtering (20 pts)

Given the following 3×3 filter, perform the image filtering using **convolution**. Give the **FULL** filtering result for each filtering process. (Hint: you need to assume an appropriate zero mapping.)

$$w = \begin{bmatrix} -1 & 0 & -1 \\ 0 & 2 & 0 \\ 1 & 0 & 1 \end{bmatrix}$$

(a) a 2×2 image as shown in image (a) (10 pts)

(b) a 3×3 image as shown in image (b) (10 pts)

1	2
3	4

(a)

0	0	0
0	1	0
0	0	0

(b)

Setup: Convolution Formula

The convolution is defined as:

$$g(x, y) = \sum_{s=-1}^1 \sum_{t=-1}^1 w(s, t) f(x - s, y - t)$$

where $f(x, y) = 0$ outside the image (zero-padding). Because the problem asks for **FULL** convolution, the output size is $(M + 2) \times (N + 2)$ for an $M \times N$ input and a 3×3 mask.

The mask w with indices $s, t \in \{-1, 0, 1\}$:

$$w(-1, -1) = -1, \quad w(-1, 0) = 0, \quad w(-1, 1) = -1$$

$$w(0, -1) = 0, \quad w(0, 0) = 2, \quad w(0, 1) = 0$$

$$w(1, -1) = 1, \quad w(1, 0) = 0, \quad w(1, 1) = 1$$

Part (a): 2×2 Image $\rightarrow 4 \times 4$ FULL Output

Image: $f(0,0) = 1$, $f(0,1) = 2$, $f(1,0) = 3$, $f(1,1) = 4$. All other $f = 0$ (zero-padded border).

The full convolution result is:

$$g_{\text{full}} = \begin{bmatrix} -1 & -2 & -1 & -2 \\ -3 & -2 & 1 & -4 \\ 1 & 8 & 9 & 2 \\ 3 & 4 & 3 & 4 \end{bmatrix}.$$

Part (a) Result Grid

-1	-2	-1	-2
-3	-2	1	-4
1	8	9	2
3	4	3	4

Part (b): 3×3 Image $\rightarrow 5 \times 5$ FULL Output

Image: $f(1,1) = 1$, all other $f(i,j) = 0$. This is a unit impulse at position $(1,1)$.

For full convolution, the impulse response appears centered in the 5×5 output support:

$$g_{\text{full}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Part (b) Result Grid

0	0	0	0	0
0	-1	0	-1	0
0	0	2	0	0
0	1	0	1	0
0	0	0	0	0

The central 3×3 block equals the filter response around the impulse location, embedded in the full 5×5 support. ■

Results

(a) The full convolution output (2×2 image with 3×3 mask) is:

$$g_{\text{full}} = \begin{bmatrix} -1 & -2 & -1 & -2 \\ -3 & -2 & 1 & -4 \\ 1 & 8 & 9 & 2 \\ 3 & 4 & 3 & 4 \end{bmatrix}$$

(b) The full convolution output (3×3 impulse image with 3×3 mask) is:

$$g_{\text{full}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & -1 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad \blacksquare$$